Examen Final de Laboratorio - Sistemas Operativos

15 de Enero de 2025

Makefile (0,5 puntos)

Crea un único Makefile que permita generar todos los programas al mismo tiempo y también cada uno de ellos por separado. Añade una regla (`clean`) para eliminar todos los archivos binarios y/o objeto.

Control de errores y Usage (0,5 puntos)

Todos los programas deben comprobar los errores de **TODAS** las llamadas al sistema (excepto `write` por la pantalla), verificar los argumentos de entrada y definir la función `usage()`.

Ejercicio 1 (2 puntos): Generador de Matrices

> NOTA: La puntuación de este ejercicio será de 2 puntos solo si funciona según las especificaciones, o 0 en caso contrario.

Escribe un programa llamado `matrixgen.c` que genere una matriz de enteros definida por el usuario. El programa debe tener dos argumentos obligatorios y un tercer argumento opcional. Los argumentos obligatorios son (i) el número de filas y (ii) el número de columnas, que deben ser enteros positivos. El tercer argumento es un nombre de archivo opcional. Si los argumentos de entrada son inválidos o faltan, el programa debe mostrar un mensaje de uso y acabar.

Una vez validados los argumentos, el programa debe reservar memoria dinámica para una matriz del tamaño especificado utilizando `malloc`. La matriz debe rellenarse con enteros secuenciales, comenzando desde 0 en orden de fila. Ejemplo de una matriz $3x3$:

```
0 1 2
3 4 5
6 7 8
```

Después de generar la matriz, el programa debe comprobar si se ha especificado un tercer argumento. Si se ha proporcionado un nombre de archivo, el programa debe escribir la matriz en el archivo en formato binario (representación interna de la máquina) con una única llamada a `write`. El archivo debe crearse con permisos `rw-r--r--` (lectura y escritura para el usuario y solo lectura para el grupo y otros). Si el archivo ya existe, debe sobrescribirse.

Si no se proporciona ningún nombre de archivo, el programa debe escribir la matriz en la salida estándar en formato binario con una única llamada a `write`. Finalmente, el programa debe liberar la memoria reservada antes de salir.

Por ejemplo, el siguiente comando crea una matriz $3x3$ y la escribe en un archivo llamado `matrix.bin`:

```
./matrixgen 3 3 matrix.bin
```

Puedes verificar el contenido del archivo utilizando el comando `xxd`:

```
xxd matrix.bin
```

Ejercicio 2 (3 puntos): Verificador de Matrices

Escribe un programa llamado `matrixver.c` que reciba dos argumentos: (i) el nombre de un archivo que contiene una matriz válida de enteros en formato binario y (ii) el número de columnas de la matriz, un entero positivo que se considera correcto. Puedes utilizar el archivo `matrix_5x5.bin` proporcionado, que contiene una matriz $5x5$, pero el programa debe funcionar con matrices de cualquier dimensión. El programa debe imitar la ejecución de la siguiente línea de comandos:

```
cat <argument 1> | od -t d4 -w<num_bytes_per_row>
```

Donde `<argument 1>` es el primer argumento del programa y `<num_bytes_per_row>` es el número de bytes por fila de la matriz, calculado utilizando el segundo argumento del programa (número de columnas).

El programa debe crear dos procesos hijos y las pipes sin nombre necesarias para comunicar información entre los procesos hijos. El primer hijo muta a `cat` sobre el nombre del archivo especificado en el primer parámetro del programa. El segundo hijo muta para ejecutar `od`, que interpreta la salida de `cat`, mostrando el contenido de la matriz en el terminal. Ten en cuenta que el contador de argumentos (`argc`) para `od` es 4. El proceso padre espera hasta que todos los hijos terminen su ejecución y luego finaliza.

Ejemplo de ejecución:

```
./matrixver matrix_5x5.bin 5
```

El programa muestra:

```
0000000           0           1           2           3           4
0000024           5           6           7           8           9
0000050          10          11          12          13          14
0000074          15          16          17          18          19
0000120          20          21          22          23          24
0000144
```

Ejercicio 3 (4 puntos): Competición de la Suma de las Diagonales de la Matriz

Escribe un programa llamado `matrixcomp.c` que reciba un parámetro: un nombre de archivo que se garantiza que contiene una matriz cuadrada M de tamaño $5x5$ almacenada en formato binario (por ejemplo, el archivo `matrix_5x5.bin` proporcionado). El programa crea dos procesos hijos que compiten por calcular la suma de los elementos diagonales de la matriz. El primer

hijo calcula la diagonal principal (es decir, comenzando en la posición $M[0][0]$), mientras que el segundo hijo calcula la diagonal secundaria (es decir, comenzando en la posición $M[4][0]$).

Cada proceso hijo debe abrir el archivo, leyendo elementos individuales (uno a uno) de las diagonales directamente del archivo. Los procesos **solo** pueden leer elementos diagonales. Antes de realizar la llamada al sistema `read` para cada elemento, los procesos hijos deben solicitar permiso al proceso padre y luego realizar una espera activa. El hijo que calcula la diagonal principal solicita permiso enviando un signal `SIGUSR1`, y el hijo que calcula la diagonal secundaria lo hace enviando un signal `SIGUSR2`. El proceso padre debe usar handlers diferentes para `SIGUSR1` y `SIGUSR2`, que envían un signal del mismo tipo de vuelta al proceso hijo correspondiente, que reanuda la ejecución y realiza la lectura. El proceso padre debe asegurarse de que las solicitudes de permiso se conceden en el orden en que se reciben.

El proceso padre espera, sin consumir recursos de CPU, solicitudes de permiso de lectura o que los hijos completen la suma de todos los elementos. Cuando un hijo completa la suma, envía un signal `SIGALRM` al padre para indicar que ha finalizado y luego termina con un código de finalización igual a la suma calculada. El proceso padre, al recibir un signal `SIGALRM`, se despierta de la espera y muestra un mensaje que incluye la posición de finalización del hijo, su PID y su código de finalización. Luego, si es necesario, el proceso padre vuelve a esperar, sin consumir recursos de CPU, por el siguiente hijo.

Ejemplo de ejecución:

```
./matrixcomp matrix_5x5.bin
```

El programa muestra:

```
First: PID 12345 with sum = 60
Second: PID 12346 with sum = 60
```

Entrega

Un único archivo `labexam.tar.gz` que contenga el Makefile y los tres programas:

```
tar -czvf labexam.tar.gz Makefile *.c
```